

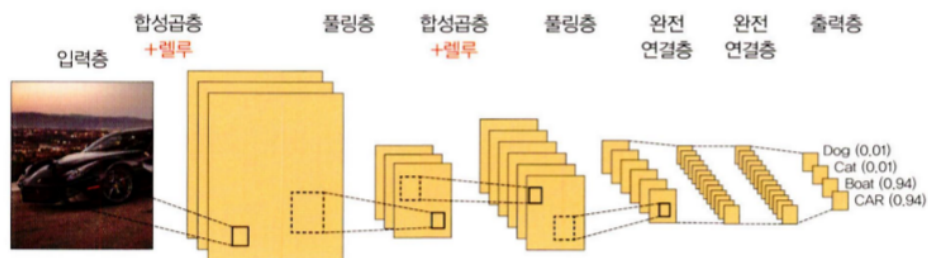
3주차_5장(1)

5장. 합성곱 신경망 I

5.1 합성곱 신경망

- 역전파 계산 과정은 1. 복잡 2. 많은 자원을 요구 3. 오래 걸림
⇒ 합성곱 신경망: 이미지의 국소적 부분을 계산하여 문제를 해결
- 이미지 분석은 배열을 펼쳐서 각 픽셀에 가중치를 곱하고 은닉층으로 전달
→ 데이터의 공간적 구조를 무시 ⇒ **합성곱층 도입**

[합성곱 신경망 구조]



- 합성곱 신경망은 다차원 배열 처리에 특화되어있다.
 - 합성곱층과 풀링층을 거치며 이미지의 주요 특성 벡터를 추출.
 - 추출된 주요 특성 벡터들은 완전연결층을 거치면서 1차원 벡터로 변환.
 - 출력층에서 소프트맥스 함수(활성화 함수)를 사용해 최종 결과 출력.

1. 입력층

: 입력 이미지 데이터가 최초로 거치게 되는 계층

- 이미지 데이터는 3차원 데이터 (높이, 너비, 채널)
 - 채널: 흑백이면 1, 컬러이면 3

2. 합성곱층

: 입력 데이터에서 특성을 추출하는 계층

- 커널(필터)로 이미지를 훑으며 특성을 추출 ⇒ **특성 맵** 도출



- 커널은 보통 3X3, 5X5 크기로 적용, 스트라이드라는 지정된 간격에 따라 순회
- 입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱해 모두 더함
- 컬러 이미지의 필터는 채널이 3으로, RGB 각각에 다른 가중치 적용
- 필터 여러 개 사용 가능, 필터의 개수 = 출력 채널 수

3. 풀링층

: 특성 맵의 차원을 다운 샘플링(이미지 축소)하는 계층

- 최대 풀링: 대상 영역에서 최댓값 추출 → 보통 사용됨
- 평균 풀링: 대상 영역에서 평균 반환

4. 완전연결층

- 합성곱층과 풀링층을 거치며 축소된 특성 맵은 완전연결층으로 전달됨.
- 이 과정에서 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐짐.

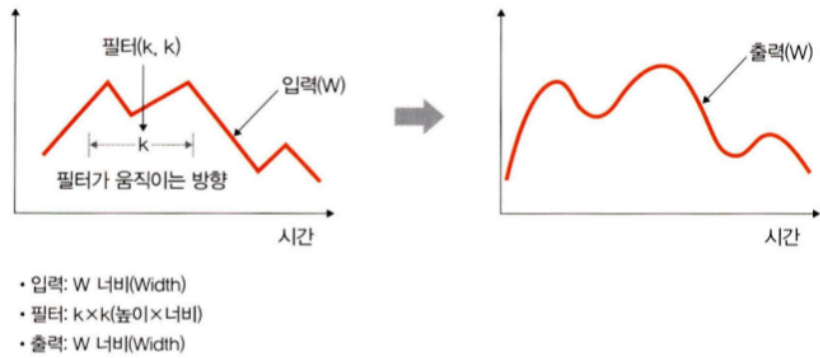
5. 출력층

- **소프트맥스 활성화 함수**를 사용해 이미지가 각 레이블에 속할 확률 출력 → 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정

[1D, 2D, 3D 합성곱]

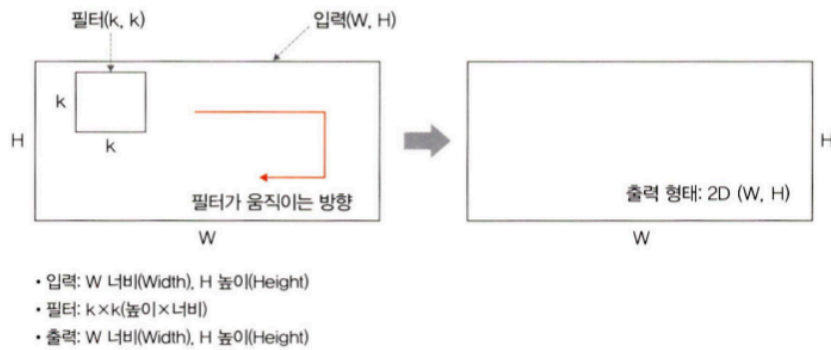
이동하는 방향의 수와 출력 형태에 따라 분류한다.

1. 1D 합성곱



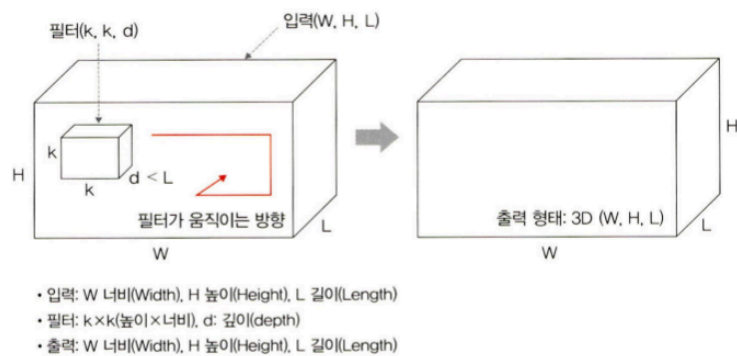
- 필터가 시간을 축으로 좌우로만 이동, 출력 형태가 1D의 배열
- 그래프 곡선을 완화할 때 사용

2. 2D 합성곱



- 필터가 방향 두 개로 이동, 출력 형태가 2D 행렬

3. 3D 합성곱



- 필터가 방향 세 개로 이동, 출력 형태가 3D
- 입력(W,H,L), 필터(k,k,d)일 때 $d < L$ 을 유지하는 것이 중요

4. 3D 입력을 갖는 2D 합성곱

- 필터의 길이가 입력 채널의 길이와 같음
- 3D 입력(W,H,L)에 필터(k,k,L) 적용 → 2D (W,H) 출력
- ex) LeNet-5, VGG

5. 1 X 1 합성곱

- 3D 입력(W,H,L)에 필터(1,1,L) 적용 → 2D (W,H) 출력
- 채널 수를 조정해 연산량이 감소
- ex) GoogLeNet

5.2 합성곱 신경망 맛보기

- fashion_mnist 데이터셋:

torchvision에 내장된 예제 데이터로, 열 가지로 분류될 수 있는 28x28 픽셀의 이미지 7만 개로 구성.

```
train_dataset=torchvision.datasets.FashionMNIST('.../chap05/data',download=True,
transform=transforms.Compose([transforms.ToTensor()]))
test_dataset=torchvision.datasets.FashionMNIST('.../chap05/data',download=True,train=False,
transform=transforms.Compose([transforms.ToTensor()])))
```

0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot

- `torchvision.datasets` : CIFAR, COCO, MNIST, ImageNet 등의 데이터셋 포함

```
torchvision.datasets.FashionMNIST(".../chap05/data", download=True,
                                     ①          ②
                                     transform=transforms.Compose([transforms.ToTensor()]))
                                     ③
```

① 첫 번째 파라미터: FashionMNIST를 내려받을 위치를 지정합니다.

② download: download를 True로 변경해 주면 첫 번째 파라미터의 위치에 해당 데이터셋이 있는지 확인한 후 내려받습니다.

③ transform: 이미지를 텐서(0~1)로 변경합니다.

- `torch.utils.data.DataLoader`: 원하는 크기의 배치 단위로 데이터를 불러오거나 순서를 무작위로 섞을 수 있음

```
torch.utils.data.DataLoader(train_dataset, batch_size=100)
```

(a) (b)

① 첫 번째 파라미터: 데이터를 불러올 데이터셋을 지정합니다.

② `batch_size`: 데이터를 배치로 묶어 줍니다. 여기에서는 `batch_size=100`으로 지정했기 때문에 100개 단위로 데이터를 묶어서 불러옵니다.

[심층 신경망]

```
# 모델 생성
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN,self).__init__()
        self.fc1=nn.Linear(in_features=784,out_features=25
6)
        self.drop=nn.Dropout(0.25)
        self.fc2=nn.Linear(in_features=256,out_features=12
8)
        self.fc3=nn.Linear(in_features=128,out_features=10)
    def forward(self,input_data):
        out=input_data.view(-1,784)    # 텐서 크기 변경
        out=F.relu(self.fc1(out))      # 활성화 함수 지정
        out=self.drop(out)
        out=F.relu(self.fc2(out))
        out=self.fc3(out)
        return out
```

- `nn.Module`을 상속 받음
- `nn.Linear(in_features=__, out_features=__)`: 단순 선형 회귀 모델
- `forward()`: 순전파 학습 (x로부터 예측된 y를 얻는 것), 객체 호출하면 자동 실행

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

```
# 파라미터 정의
learning_rate=0.001      # 학습률
model=FashionDNN()
model.to(device)
criterion=nn.CrossEntropyLoss() # 손실함수
optimizer=torch.optim.Adam(model.parameters(),lr=learning_rate) # 옵티마이저
```

```
# 모델 학습 및 성능 평가
num_epochs=5
count=0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader: # 레코드 하나씩 가져옴
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels) # 자동 미분 사용하기 위함

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
```

```

        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0

        for images, labels in test_loader:
            images, labels = images.to(device), labels.
to(device)

            labels_list.append(labels)

            test = Variable(images.view(100, 1, 28, 2
8))

            outputs = model(test)

            predictions = torch.max(outputs, 1)[1].to(d
evice)

            predictions_list.append(predictions)

            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total

        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    if not (count % 500):
        print(
            "Iteration: {}, Loss: {}, Accuracy: {}".fo
rmat(
                count, loss.data, accuracy

```

```
)  
)
```

[합성곱 신경망]

```
# 모델 생성  
class FashionCNN(nn.Module):  
    def __init__(self):  
        super(FashionCNN, self).__init__()  
  
        self.layer1 = nn.Sequential(  
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),  
            nn.BatchNorm2d(32),  
            nn.ReLU(),  
            nn.MaxPool2d(kernel_size=2, stride=2))  
  
        self.layer2 = nn.Sequential(  
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),  
            nn.BatchNorm2d(64),  
            nn.ReLU(),  
            nn.MaxPool2d(2))  
  
        self.fc1 = nn.Linear(in_features=64 * 6 * 6, out_features=600)  
        self.drop = nn.Dropout2d(0.25)  
        self.fc2 = nn.Linear(in_features=600, out_features=120)  
        self.fc3 = nn.Linear(in_features=120, out_features=10)  
  
    def forward(self, x):  
        out = self.layer1(x)  
        out = self.layer2(out)  
        out = out.view(out.size(0), -1)  
        out = self.fc1(out)  
        out = self.drop(out)
```



```

        out = self.fc2(out)
        out = self.fc3(out)
        return out

# 파라미터 정의
learning_rate = 0.001
model = FashionCNN()

model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

print(model)

```

- `nn.Sequential` : forward()에서 구현될 순전파를 계층 형태로 작성할 수 있음
- `nn.Conv2d(in_channels=1,out_channels=32,kernel_size=3,padding=1)` : 합성곱층
 - kernel_size=3이라 하면 (3,3)를 의미하고, (3,5) 등으로 지정할수도 있음
 - padding: 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채움
- `nn.BatchNorm2d(32)` : 각 배치를 정규화함
- `nn.MaxPool2d(kernel_size=2, stride=2)` : 풀링 계층